

AssetDesk — Technical Documentation

Documentation set for the AssetDesk Forge app. Written for a technical team member (or future maintainer) who has never seen this codebase. Start here, then follow the links below in order.

Last updated: 2026-07-06 · App version at time of writing: 6.12.0 (development)

What is AssetDesk?

AssetDesk is an **Atlassian Forge app** for Jira Service Management (JSM). It solves one problem: *JSM Assets (the CMDB) is only visible to licensed agents — portal customers can't see the assets assigned to them.*

AssetDesk fixes that:

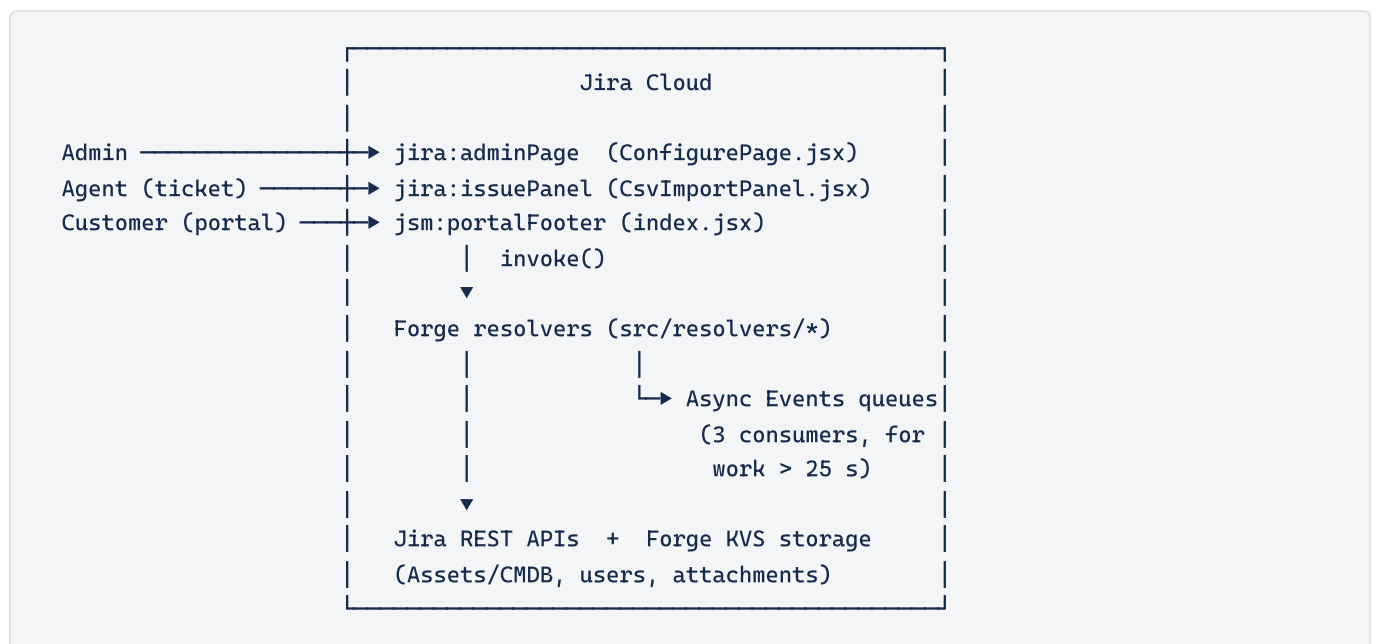
1. A **Jira admin** configures which Assets schema to expose, which attributes are visible, and — most importantly — **AQL "ownership" rules** that define what "my assets" means (e.g. `Owner = <the logged-in user's displayName>`).
2. **Portal customers and agents** then see a filtered, paginated, searchable table of *their own* assets in the JSM portal footer — with optional inline editing and XLSX/PDF export.
3. **Agents** additionally get a CSV import panel on tickets: attach a CSV, and each row is created or updated as an Assets object.

It is a **single-customer installation** (not Marketplace-distributed) running on Forge with **UI Kit** (`@forge/react`) — no custom UI, no iframes.

Documentation index

Doc	What it covers	Read it when...
architecture.md	Infrastructure, module map, technology choices, all data-flow diagrams (asset load, edit, export, CSV import, admin config)	You want the big picture of how the pieces connect
backend.md	Every resolver and queue consumer, function by function, with code snippets explaining what each does and why	You're changing server-side behavior
frontend.md	The four UI entry points (portal footer, Configure page, Get Started page, CSV import panel), component by component	You're changing what users see
review.md	Honest quality assessment, strengths, weaknesses, and a prioritized list of improvement suggestions	You're planning what to do next

30-second architecture summary



Three things to internalize before touching anything:

1. **The 25-second rule.** Synchronous `invoke()` resolver calls are hard-killed at 25 s by the Forge platform. Anything potentially slow (asset loading, file export, CSV import) runs as an **async queue job**: a tiny resolver enqueues, a consumer function does the work (up to 900 s), the frontend polls a KVS key for the result. See [architecture.md § Async job pattern](#).
2. **AQL is the security boundary.** "Which assets belong to this user" is decided *server-side* by AQL queries built from admin-configured rules plus the caller's verified identity. Queue consumers always run `asApp()` (no user session exists there) — the AQL ownership filter, not the auth mode, is what scopes results.
3. **KVS has a 128 KB per-value cap.** Every cache/job-result design decision in the codebase (slim cache stubs, capped error lists, chunked export payloads) exists because of this limit.

Repository layout

```
AssetDesk/
├─ manifest.yml          # Forge app definition: modules, functions, scopes
├─ package.json          # deps: @forge/*, xlsx, pdfkit, react
├─ CLAUDE.md / AGENTS.md # working agreements for AI-assisted development
├─ docs/                 # ← you are here
└─ src/
    ├─ index.js          # re-exports resolver handler (manifest entry)
    ├─ assetLoadConsumer.js # thin re-export → resolvers/assetLoadConsumer
    ├─ csvImportConsumer.js # thin re-export → resolvers/csvImportConsumer
    ├─ exportJobConsumer.js # thin re-export → resolvers/exportJobConsumer
    ├─ frontend/
    │   ├─ index.jsx      # PORTAL FOOTER – the main "My Assets" view (~2,300 lines)
    │   ├─ ConfigurePage.jsx # ADMIN – schema/AQL/visibility config (~2,400 lines)
    │   ├─ GetStartedPage.jsx # ADMIN – onboarding walkthrough
    │   └─ CsvImportPanel.jsx # AGENT – CSV import issue panel
    ├─ resolvers/
    │   ├─ index.js       # portal-footer resolvers + async job start/poll
    │   ├─ shared.js      # THE core library: AQL, search, cache, payload building
    │   ├─ adminResolvers.js # admin-page resolvers (schemas, save, reconcile)
    │   ├─ exportAssets.js # XLSX/PDF builders + export job resolvers
    │   ├─ assetLoadConsumer.js # queue consumer: the heavy asset fetch
    │   ├─ csvImport.js    # CSV import resolvers (preview, start, poll)
    │   ├─ csvImportConsumer.js # queue consumer: row-by-row create/update
    │   └─ exportJobConsumer.js # queue consumer: build XLSX/PDF, chunk into KVS
    └─ resources/         # images for GetStartedPage, issue-panel icon
```

Where's the entry point? `manifest.yml` maps each UI module to a resource file and every module's `resolver.function` to `index.handler` — which is `src/index.js` → `src/resolvers/index.js` → `resolver.getDefinitions()`. One `Resolver` instance; definitions are split across files purely for organization (`registerAdminResolvers(resolver)`, `registerExportAssets(resolver)`, `registerCsvImportResolvers(resolver)` are called at the bottom of `src/resolvers/index.js`).

Developer quickstart

```
npm install          # once
forge lint           # validate manifest + code
forge deploy --non-interactive -e development
forge install --non-interactive --upgrade \
  --site <your-site>.atlassian.net --product jira --environment development
forge tunnel         # local dev loop (restart after manifest changes)
forge logs -n 100 -e development # tail backend logs
```

Rules that will bite you if you skip them:

- **UI components must come from `@forge/react`** — never plain `<div>` / `<span>`, never `@forge/ui` (deprecated), never third-party component libraries. There is no `Table` component; use `DynamicTable`.

- After **any** `manifest.yml` change: `forge lint`, redeploy, and if scopes or modules changed, `forge install --upgrade` too.
- There are no automated tests (see [review.md](#)) — verification is manual against the development site.